

Árvores Binárias

Diego Padilha Rubert

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Estruturas de Dados

- ▶ As estruturas de dados estudadas até o momento são sequenciais: pilhas, filas, listas, tabelas de dispersão, listas de prioridades (heaps)
- ▶ Começaremos a estudar **árvores**, estruturas de dados mais complexas, utilizadas em variadas aplicações

- ▶ As estruturas de dados estudadas até o momento são sequenciais: pilhas, filas, listas, tabelas de dispersão, listas de prioridades (heaps)
- ▶ Começaremos a estudar **árvores**, estruturas de dados mais complexas, utilizadas em variadas aplicações

Uma árvore T é um conjunto finito de elementos chamados *nós* ou *vértices*, de forma que:

- ▶ $T = \emptyset$, ou seja, temos uma árvore *vazia*, ou

- ▶ Existe um nó r chamado *raiz* de T , e:

- ▶ r é o único nó de T , ou

- ▶ r tem pelo menos um filho, e cada filho de r é a raiz de uma árvore, e cada uma por sua vez uma árvore...

Uma árvore T é um conjunto finito de elementos chamados *nós* ou *vértices*, de forma que:

- ▶ $T = \emptyset$, ou seja, temos uma árvore *vazia*, ou
- ▶ Existe um nó r chamado *raiz* de T , e:
 - ▶ r é o único nó de T , ou
 - ▶ os demais são divididos em até $m \geq 1$ conjuntos disjuntos não vazios, as *subárvores* de r , cada qual por sua vez uma árvore

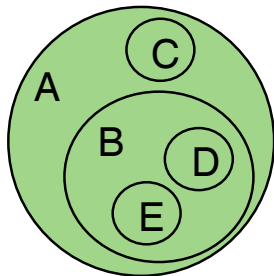
Uma árvore T é um conjunto finito de elementos chamados *nós* ou *vértices*, de forma que:

- ▶ $T = \emptyset$, ou seja, temos uma árvore *vazia*, ou
- ▶ Existe um nó r chamado *raiz* de T , e:
 - ▶ r é o único nó de T , ou
 - ▶ os demais são divididos em até $m \geq 1$ conjuntos disjuntos não vazios, as *subárvores* de r , cada qual por sua vez uma árvore

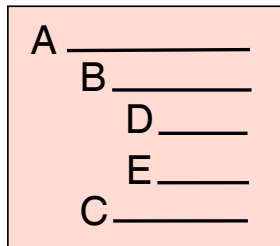
Uma árvore T é um conjunto finito de elementos chamados *nós* ou *vértices*, de forma que:

- ▶ $T = \emptyset$, ou seja, temos uma árvore *vazia*, ou
- ▶ Existe um nó r chamado *raiz* de T , e:
 - ▶ r é o único nó de T , ou
 - ▶ os demais são divididos em até $m \geq 1$ conjuntos disjuntos não vazios, as *subárvores* de r , cada qual por sua vez uma árvore

Árvores - Representações



a)



b)

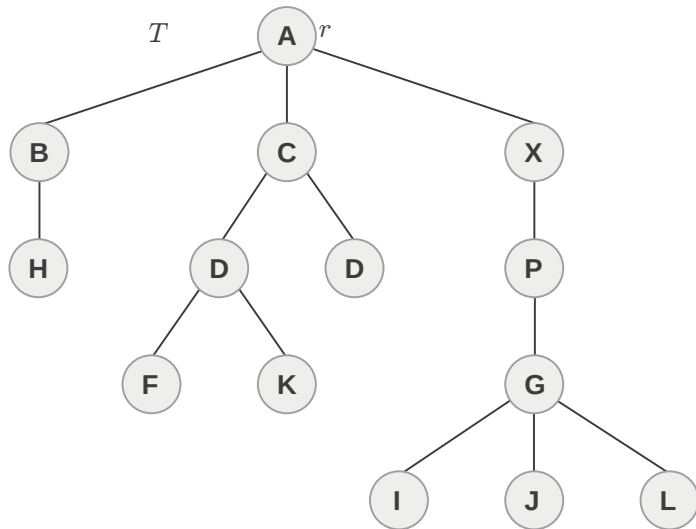
1A; 1.1B; 1.1.1D; 1.1.2E; 1.2C

c)

(A(B(D)(E))(C))

d)

Árvores - Representação padrão



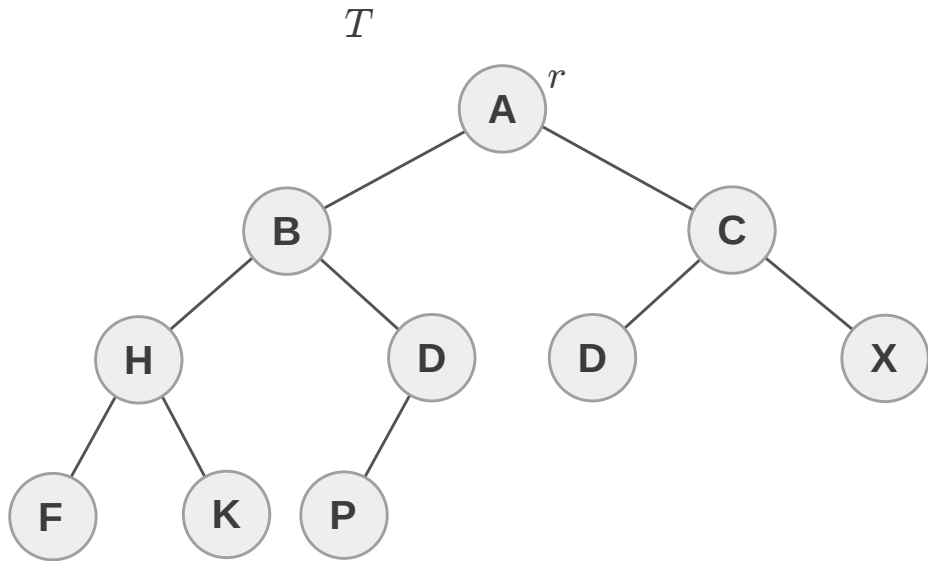
Caso especial onde $1 \leq m \leq 2$:

- ▶ $T = \emptyset$, ou seja, temos uma árvore *vazia*, ou
- ▶ Existe um nó r chamado *raiz* de T , e:
 - ▶ r é o único nó de T , ou
 - ▶ os demais são divididos em até $1 \leq m \leq 2$ conjuntos disjuntos não vazios, as *subárvores* de r , cada qual por sua vez uma árvore

Caso especial onde $1 \leq m \leq 2$:

- ▶ $T = \emptyset$, ou seja, temos uma árvore *vazia*, ou
- ▶ Existe um nó r chamado *raiz* de T , e:
 - ▶ r é o único nó de T , ou
 - ▶ os demais são divididos em até $1 \leq m \leq 2$ conjuntos disjuntos não vazios, as *subárvores* de r , cada qual por sua vez uma árvore

Representação



Seja um nó v :

- ▶ T_v é a subárvore de T com raiz v
- ▶ Nós raízes u e w das subárvores de T_v (caso existam) são chamados *filhos* de v
- ▶ v é chamado *pai* de u e w
- ▶ Os nós u e w são *irmãos*

* esses e demais conceitos se estendem para árvores não binárias

Seja um nó v :

- ▶ T_v é a subárvore de T com raiz v
- ▶ Nós raízes u e w das subárvores de T_v (caso existam) são chamados *filhos* de v
- ▶ v é chamado *pai* de u e w
- ▶ Os nós u e w são *irmãos*

* esses e demais conceitos se estendem para árvores não binárias

Seja um nó v :

- ▶ T_v é a subárvore de T com raiz v
- ▶ Nós raízes u e w das subárvores de T_v (caso existam) são chamados *filhos* de v
- ▶ v é chamado *pai* de u e w
- ▶ Os nós u e w são *irmãos*

* esses e demais conceitos se estendem para árvores não binárias

Seja um nó v :

- ▶ T_v é a subárvore de T com raiz v
- ▶ Nós raízes u e w das subárvores de T_v (caso existam) são chamados *filhos* de v
- ▶ v é chamado *pai* de u e w
- ▶ Os nós u e w são *irmãos*

* esses e demais conceitos se estendem para árvores não binárias

Seja um nó v :

- ▶ T_v é a subárvore de T com raiz v
- ▶ Nós raízes u e w das subárvores de T_v (caso existam) são chamados *filhos* de v
- ▶ v é chamado *pai* de u e w
- ▶ Os nós u e w são *irmãos*

* esses e demais conceitos se estendem para árvores não binárias

Seja um nó v :

- ▶ T_v é a subárvore de T com raiz v
- ▶ Nós raízes u e w das subárvores de T_v (caso existam) são chamados *filhos* de v
- ▶ v é chamado *pai* de u e w
- ▶ Os nós u e w são *irmãos*

* esses e demais conceitos se estendem para árvores não binárias

Conceitos (2)

- ▶ Um nó que não tem filhos é uma *folha*
- ▶ Um *caminho* na árvore é uma sequência de nós distintos $v_1, \dots, v_k$ tal que existe sempre entre nós consecutivos a relação “é pai de” ou “é filho de”
- ▶ Dado um nó v , nós que estão no caminho de v até a raiz são *ancestrais* ou *ascendentes* (incluindo a raiz)
- ▶ Dado um nó v , nós que estão em um caminho de v até uma folha são *descendentes* (incluindo a folha)
- ▶ Dado um nó v , T_v^E e T_v^D são suas *subárvores esquerda e direita*

Conceitos (2)

- ▶ Um nó que não tem filhos é uma *folha*
- ▶ Um *caminho* na árvore é uma sequência de nós distintos $v_1, \dots, v_k$ tal que existe sempre entre nós consecutivos a relação “é pai de” ou “é filho de”
- ▶ Dado um nó v , nós que estão no caminho de v até a raiz são *ancestrais* ou *ascendentes* (incluindo a raiz)
- ▶ Dado um nó v , nós que estão em um caminho de v até uma folha são *descendentes* (incluindo a folha)
- ▶ Dado um nó v , T_v^E e T_v^D são suas *subárvores esquerda e direita*

Conceitos (2)

- ▶ Um nó que não tem filhos é uma *folha*
- ▶ Um *caminho* na árvore é uma sequência de nós distintos $v_1, \dots, v_k$ tal que existe sempre entre nós consecutivos a relação “é pai de” ou “é filho de”
- ▶ Dado um nó v , nós que estão no caminho de v até a raiz são *ancestrais* ou *ascendentes* (incluindo a raiz)
- ▶ Dado um nó v , nós que estão em um caminho de v até uma folha são *descendentes* (incluindo a folha)
- ▶ Dado um nó v , T_v^E e T_v^D são suas *subárvores esquerda e direita*

Conceitos (2)

- ▶ Um nó que não tem filhos é uma *folha*
- ▶ Um *caminho* na árvore é uma sequência de nós distintos $v_1, \dots, v_k$ tal que existe sempre entre nós consecutivos a relação “é pai de” ou “é filho de”
- ▶ Dado um nó v , nós que estão no caminho de v até a raiz são *ancestrais* ou *ascendentes* (incluindo a raiz)
- ▶ Dado um nó v , nós que estão em um caminho de v até uma folha são *descendentes* (incluindo a folha)
- ▶ Dado um nó v , T_v^E e T_v^D são suas *subárvores esquerda e direita*

Conceitos (2)

- ▶ Um nó que não tem filhos é uma *folha*
- ▶ Um *caminho* na árvore é uma sequência de nós distintos $v_1, \dots, v_k$ tal que existe sempre entre nós consecutivos a relação “é pai de” ou “é filho de”
- ▶ Dado um nó v , nós que estão no caminho de v até a raiz são *ancestrais* ou *ascendentes* (incluindo a raiz)
- ▶ Dado um nó v , nós que estão em um caminho de v até uma folha são *descendentes* (incluindo a folha)
- ▶ Dado um nó v , T_v^E e T_v^D são suas *subárvores esquerda e direita*

Conceitos (3)

- ▶ O número de *saltos* ou *passos* de um caminho com n nós é $n - 1$
- ▶ O *nível* (ou *profundidade*) de um nó v é o número de saltos de r até v (o nível da raiz é 0)
- ▶ A *altura* $h(v)$ de um nó v é o maior número de saltos de v até uma folha descendente (a altura das folhas é 0)
- ▶ A *altura* de uma árvore T é a altura da raiz r , ou seja, $h(T) = h(r)$

Conceitos (3)

- ▶ O número de *saltos* ou *passos* de um caminho com n nós é $n - 1$
- ▶ O *nível* (ou *profundidade*) de um nó v é o número de saltos de r até v (o nível da raiz é 0)
- ▶ A *altura* $h(v)$ de um nó v é o maior número de saltos de v até uma folha descendente (a altura das folhas é 0)
- ▶ A *altura* de uma árvore T é a altura da raiz r , ou seja, $h(T) = h(r)$

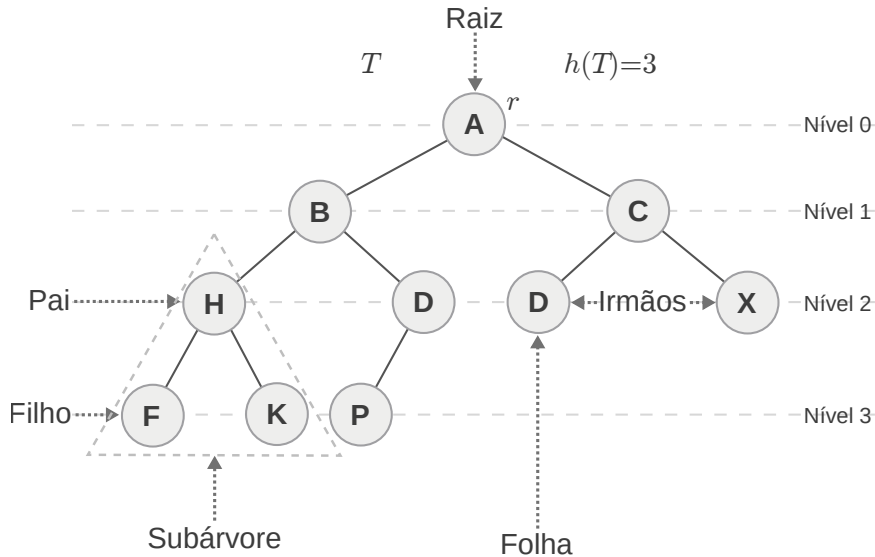
Conceitos (3)

- ▶ O número de *saltos* ou *passos* de um caminho com n nós é $n - 1$
- ▶ O *nível* (ou *profundidade*) de um nó v é o número de saltos de r até v (o nível da raiz é 0)
- ▶ A *altura* $h(v)$ de um nó v é o maior número de saltos de v até uma folha descendente (a altura das folhas é 0)
- ▶ A *altura* de uma árvore T é a altura da raiz r , ou seja, $h(T) = h(r)$

Conceitos (3)

- ▶ O número de *saltos* ou *passos* de um caminho com n nós é $n - 1$
- ▶ O *nível* (ou *profundidade*) de um nó v é o número de saltos de r até v (o nível da raiz é 0)
- ▶ A *altura* $h(v)$ de um nó v é o maior número de saltos de v até uma folha descendente (a altura das folhas é 0)
- ▶ A *altura* de uma árvore T é a altura da raiz r , ou seja, $h(T) = h(r)$

Conceitos (4)



Árvores binárias particulares

- ▶ **Árvore estritamente binária:** cada nó possui 0 ou 2 filhos
- ▶ Árvore binária completa: todos os níveis têm o máximo número de elementos, exceto possivelmente o último
- ▶ Árvore zigue-zague ou degenerada: todo nó exatamente 1 filho, exceto a folha
- ▶ Dada uma quantidade n de nós, a árvore contendo n nós com a maior altura possível é a árvore zigue-zague

Árvores binárias particulares

- ▶ Árvore estritamente binária: cada nó possui 0 ou 2 filhos
- ▶ Árvore binária completa: todos os níveis têm o máximo número de elementos, exceto possivelmente o último
- ▶ Árvore zigue-zague ou degenerada: todo nó exatamente 1 filho, exceto a folha
- ▶ Dada uma quantidade n de nós, a árvore contendo n nós com a maior altura possível é a árvore zigue-zague

Árvores binárias particulares

- ▶ Árvore estritamente binária: cada nó possui 0 ou 2 filhos
- ▶ Árvore binária completa: todos os níveis têm o máximo número de elementos, exceto possivelmente o último
- ▶ Árvore zigue-zague ou degenerada: todo nó exatamente 1 filho, exceto a folha
- ▶ Dada uma quantidade n de nós, a árvore contendo n nós com a maior altura possível é a árvore zigue-zague

Árvores binárias particulares

- ▶ Árvore estritamente binária: cada nó possui 0 ou 2 filhos
- ▶ Árvore binária completa: todos os níveis têm o máximo número de elementos, exceto possivelmente o último
- ▶ Árvore zigue-zague ou degenerada: todo nó exatamente 1 filho, exceto a folha
- ▶ Dada uma quantidade n de nós, a árvore contendo n nós com a maior altura possível é a árvore zigue-zague

Percursos em árvores binárias

- ▶ Um *percurso* é uma visitação sistemática aos nós de uma árvore
- ▶ *Visitar* um nó é acessá-lo e verificar e/ou manipular de alguma forma as informações contidas nele (p. ex. verificar ou escrever qual é o valor armazenado)
- ▶ Os percursos se caracterizam pela ordem com que os nós são visitados
- ▶ Uma árvore binária pode ser percorrida de muitas maneiras diferentes

Percursos em árvores binárias

- ▶ Um *percurso* é uma visitação sistemática aos nós de uma árvore
- ▶ *Visitar* um nó é acessá-lo e verificar e/ou manipular de alguma forma as informações contidas nele (p. ex. verificar ou escrever qual é o valor armazenado)
- ▶ Os percursos se caracterizam pela ordem com que os nós são visitados
- ▶ Uma árvore binária pode ser percorrida de muitas maneiras diferentes

Percursos em árvores binárias

- ▶ Um *percurso* é uma visitação sistemática aos nós de uma árvore
- ▶ *Visitar* um nó é acessá-lo e verificar e/ou manipular de alguma forma as informações contidas nele (p. ex. verificar ou escrever qual é o valor armazenado)
- ▶ Os percursos se caracterizam pela ordem com que os nós são visitados
- ▶ Uma árvore binária pode ser percorrida de muitas maneiras diferentes

Percursos em árvores binárias

- ▶ Um *percurso* é uma visitação sistemática aos nós de uma árvore
- ▶ *Visitar* um nó é acessá-lo e verificar e/ou manipular de alguma forma as informações contidas nele (p. ex. verificar ou escrever qual é o valor armazenado)
- ▶ Os percursos se caracterizam pela ordem com que os nós são visitados
- ▶ Uma árvore binária pode ser percorrida de muitas maneiras diferentes

Percursos em árvores binárias

Dada a raiz r de uma (sub)árvore:

- ▶ Percurso *pré-ordem* ou *r-e-d* ●:
visita $r \rightarrow$ percorre T_r^E pré-ordem $\rightarrow$ percorre T_r^D pré-ordem
- ▶ Percurso *pós-ordem* ou *e-d-r* ●:
percorre T_r^E pós-ordem $\rightarrow$ percorre T_r^D pós-ordem $\rightarrow$ visita r
- ▶ Percurso *em-ordem* ou *e-r-d* ●:
percorre T_r^E em-ordem $\rightarrow$ visita $r \rightarrow$ percorre T_r^D em-ordem
- ▶ Note que, dado um nó v , percorremos sempre T_v^E e posteriormente T_v^D
- ▶ A diferença entre os percursos é **quando visitamos** v :



Percursos em árvores binárias

Dada a raiz r de uma (sub)árvore:

- ▶ Percurso *pré-ordem* ou *r-e-d* ●:
visita $r \rightarrow$ percorre T_r^E pré-ordem $\rightarrow$ percorre T_r^D pré-ordem
- ▶ Percurso *pós-ordem* ou *e-d-r* ●:
percorre T_r^E pós-ordem $\rightarrow$ percorre T_r^D pós-ordem $\rightarrow$ visita r
- ▶ Percurso *em-ordem* ou *e-r-d* ●:
percorre T_r^E em-ordem $\rightarrow$ visita $r \rightarrow$ percorre T_r^D em-ordem
- ▶ Note que, dado um nó v , percorremos sempre T_v^E e posteriormente T_v^D
- ▶ A diferença entre os percursos é **quando visitamos** v :



Percursos em árvores binárias

Dada a raiz r de uma (sub)árvore:

- ▶ Percurso *pré-ordem* ou *r-e-d* ●:
visita $r \rightarrow$ percorre T_r^E pré-ordem $\rightarrow$ percorre T_r^D pré-ordem
- ▶ Percurso *pós-ordem* ou *e-d-r* ●:
percorre T_r^E pós-ordem $\rightarrow$ percorre T_r^D pós-ordem $\rightarrow$ visita r
- ▶ Percurso *em-ordem* ou *e-r-d* ●:
percorre T_r^E em-ordem $\rightarrow$ visita $r \rightarrow$ percorre T_r^D em-ordem
- ▶ Note que, dado um nó v , percorremos sempre T_v^E e posteriormente T_v^D
- ▶ A diferença entre os percursos é **quando visitamos** v :



Percursos em árvores binárias

Dada a raiz r de uma (sub)árvore:

- ▶ Percurso *pré-ordem* ou *r-e-d* ●:
visita $r \rightarrow$ percorre T_r^E pré-ordem $\rightarrow$ percorre T_r^D pré-ordem
- ▶ Percurso *pós-ordem* ou *e-d-r* ●:
percorre T_r^E pós-ordem $\rightarrow$ percorre T_r^D pós-ordem $\rightarrow$ visita r
- ▶ Percurso *em-ordem* ou *e-r-d* ●:
percorre T_r^E em-ordem $\rightarrow$ visita $r \rightarrow$ percorre T_r^D em-ordem
- ▶ Note que, dado um nó v , percorremos sempre T_v^E e posteriormente T_v^D
- ▶ A diferença entre os percursos é **quando visitamos** v :



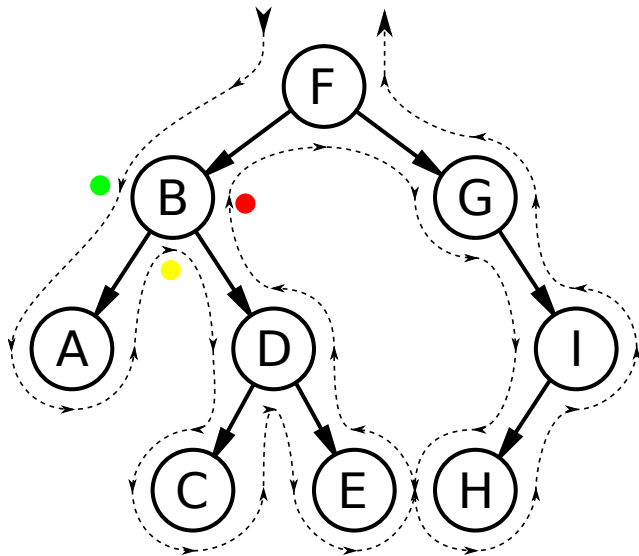
Percursos em árvores binárias

Dada a raiz r de uma (sub)árvore:

- ▶ Percurso *pré-ordem* ou *r-e-d* ●:
visita $r \rightarrow$ percorre T_r^E pré-ordem $\rightarrow$ percorre T_r^D pré-ordem
- ▶ Percurso *pós-ordem* ou *e-d-r* ●:
percorre T_r^E pós-ordem $\rightarrow$ percorre T_r^D pós-ordem $\rightarrow$ visita r
- ▶ Percurso *em-ordem* ou *e-r-d* ●:
percorre T_r^E em-ordem $\rightarrow$ visita $r \rightarrow$ percorre T_r^D em-ordem
- ▶ Note que, dado um nó v , percorremos sempre T_v^E e posteriormente T_v^D
- ▶ A diferença entre os percursos é **quando visitamos** v :



Percursos em árvores binárias



- 4.1 Escreva 3 funções PRÉ-ORDEM, PÓS-ORDEM e EM-ORDEM que, dada uma árvore binária T , escreva os elementos na ordem em que são visitados em cada um dos percursos (dica: escreva algoritmos recursivos)
- 4.2 Escreva uma função NÚMERO DE NÓS que, dada uma árvore binária T , calcule e devolva o número de nós de T
- 4.3 Escreva uma função FOLHAS que, dada uma árvore binária T , imprima em um percurso em-ordem (*e-r-d*) os conteúdos das folhas de T
- 4.4 Escreva uma função ESTÁ-EM que, dada uma árvore binária T e um valor x , devolva VERDADEIRO se T possui algum nó com valor x , e FALSO caso contrário

- 4.5 Escreva uma versão **não recursiva** da função PRÉ-ORDEM de percurso na árvore.
(Dica: use uma pilha)
- 4.6 Versões **não recursivas** das funções PÓS-ORDEM e EM-ORDEM não são triviais:
investigue como elas podem ser implementadas

